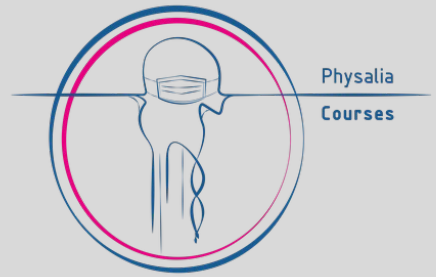
Processing NGS data

Epigenomics Data Analysis

Jacques Serizay

Physalia 2026



NGS processing workflow

- Get .bcl files
- Create fastq files
- QC: remove/trim low quality reads
- Align fastq to BAM
- Filter duplicates, artifacts, …
- Generate tracks
- Assay-specific downstream analysis

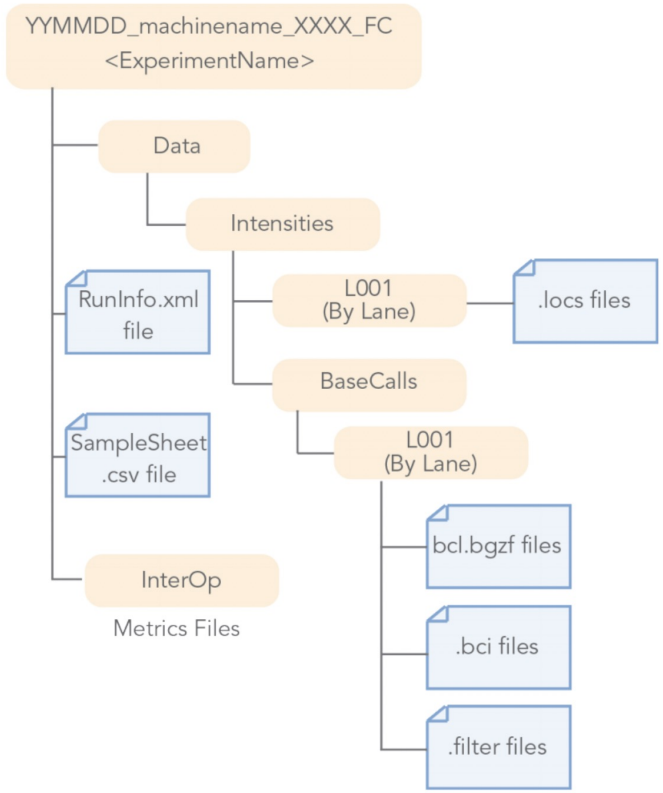
.bcl files

.bcl:

- **Raw data** output of a sequencing run
- **Binary**, non-human-readable file

- Contains the **base calling** and **quality score per cluster, per sequencing lane, per cycle**

- **Huge files**
- **No aggregated sequence per read**



NGS processing workflow

- Get .bcl files 
- Create fastq files
- QC: remove/trim low quality reads
- Align fastq to BAM
- Filter duplicates, artifacts, …
- Generate tracks
- Assay-specific downstream analysis

Fastq files

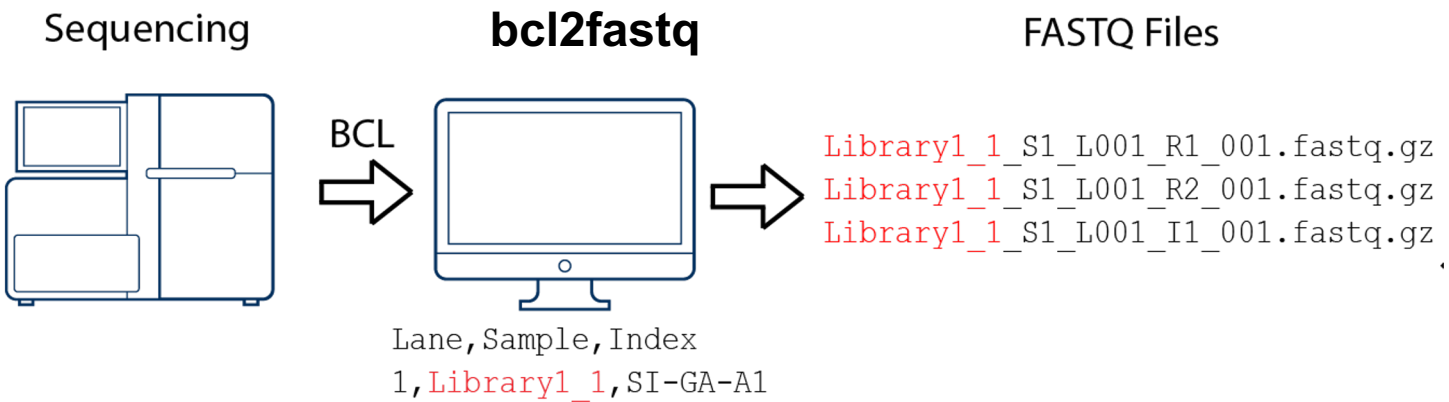
A fastq file contains reads, each read is composed of 4 lines:

1. A sequence identifier with information about the sequencing run
2. The sequence (the base calls; A, C, T, G and N).
3. A separator, which is simply a plus (+) sign.
4. The base call quality scores, using ASCII characters to represent the numerical quality scores.

```
jacquesserizay@LOCAL[12:46:19]:~ $ cat SRR11575369_1.fastq.gz | zcat |  head -n 8
@SRR11575369.1 1/1
ANCAACAGTGGAATTCGTTTTGATGAAAAAAATAAATTTGTTCTTCAAAGCAGAGTGAATGATGCAGTACGAGCTCTTGCTCTTGAAAACCCATCACAACTTTATAATTTAATAATTAGTGAAAAATTAAAAAAATAATATTTCTTATATT
+
F#F:FFFFF:FFFFFFFFFF:F,FF,FF::F,::FFFFFFFFFFF:FF:FF:F,FFF,F:F,FFF,FF:FF:F:FFF:FF:FFFF,:F::FFFF:FF,FF:F:FF,,:F:F,::,F:F:FF,,::,FF:,,F,,,,,:FFFF::F:,,,FF
@SRR11575369.2 2/1
TNGCCAGTCAATACCGCCCTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTTGTGTTTTTTTTTTTTTATATAATAAAATTTTTTTTTTTTTTAATAAAATTTTTTTTTTTATTTT
+
F#FFFFFFF::FF:FF::FFFFFFFFFFFFFFFF:FFFFFFFFFFFFF,,FFFFFFFFF:FFFFFF:FFFFFFFFFFF:F,FFFFF,,,:,,::FFFFFFFF,,:,,,F,,,,,,FFFFFF,F,,FF:,F,:,F,FFFFF,,,:,,F::F
```

bcl2fastq

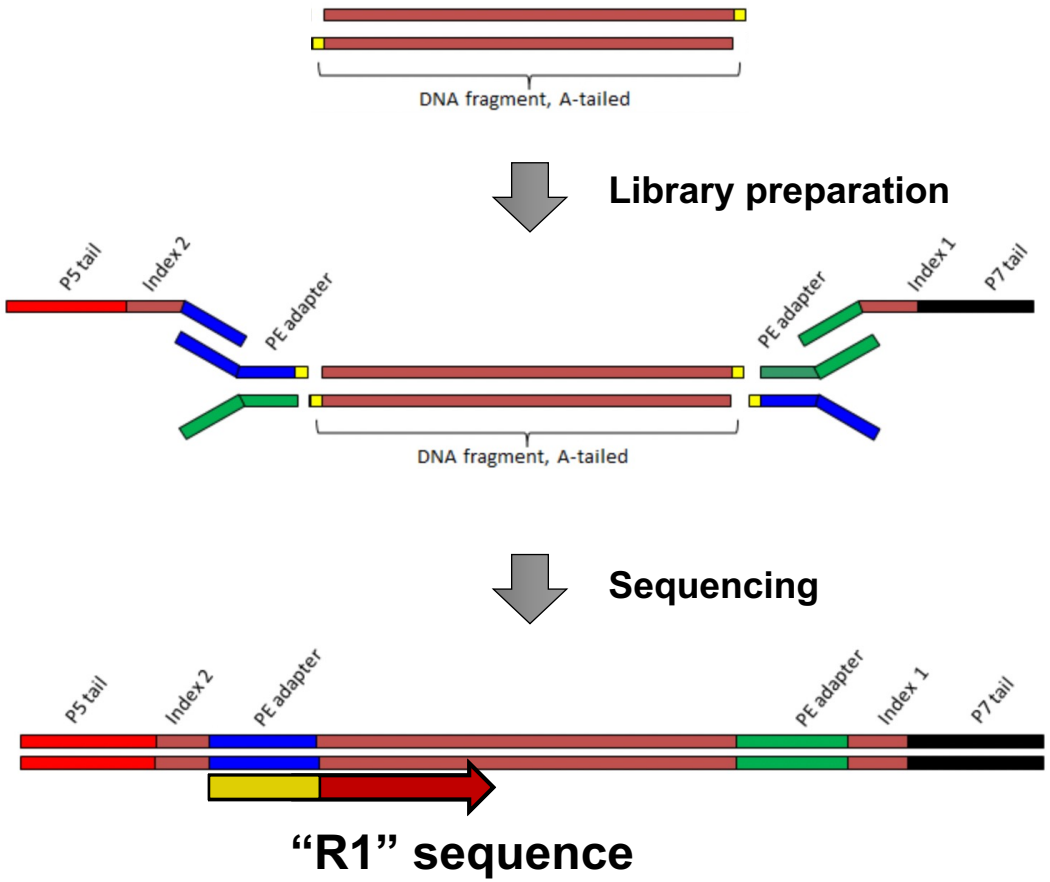
```
bcl2fastq --run-folder-dir <bcl_files_folder> --output-dir <fastq_files_folder>
```



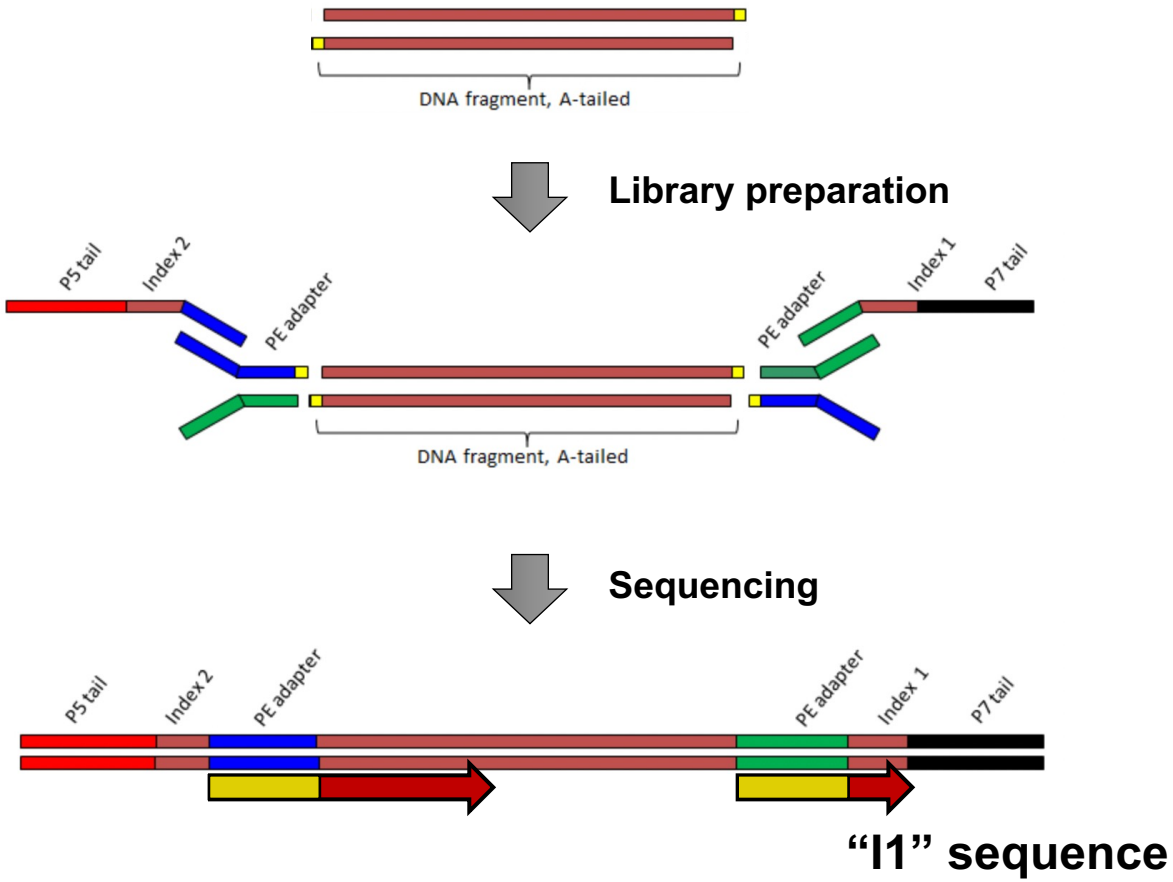
User guide:

https://support.illumina.com/content/dam/illumina-support/documents/documentation/software_documentation/bcl2fastq/bcl2fastq_letterbooklet_15038058brpmi.pdf

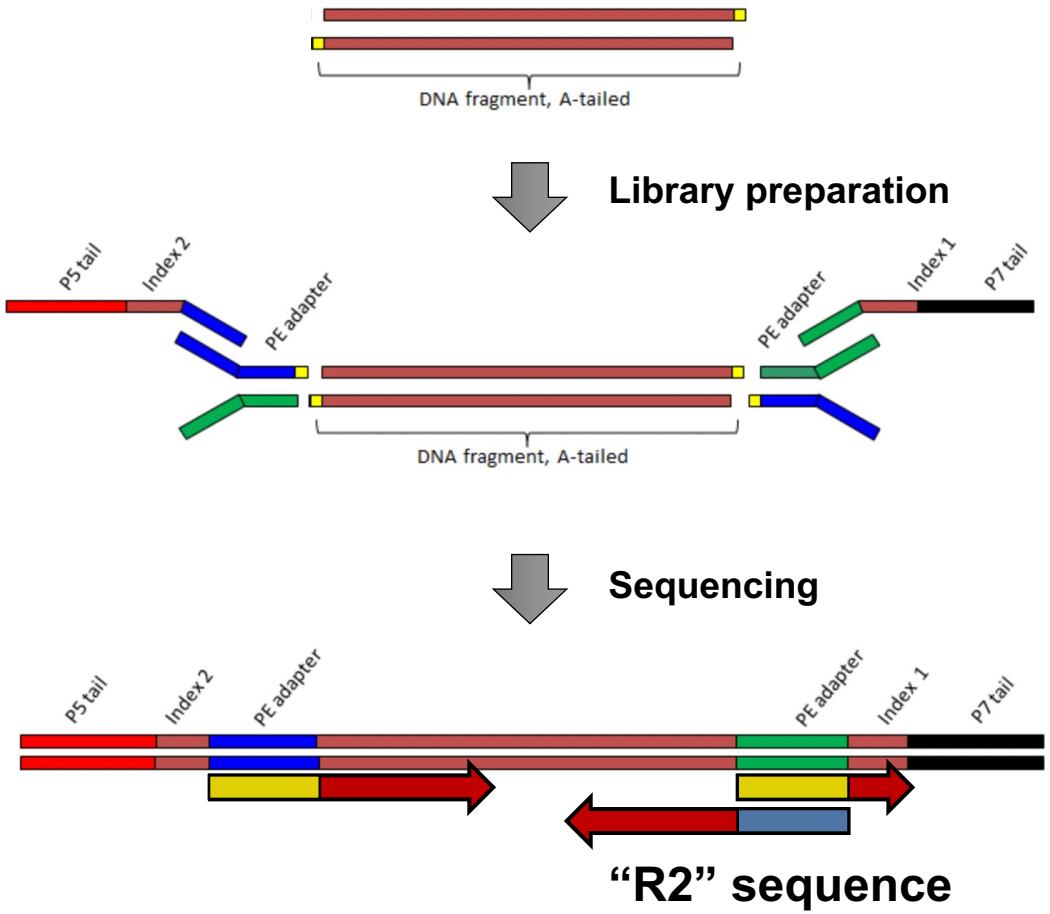
Why so many fastq files?



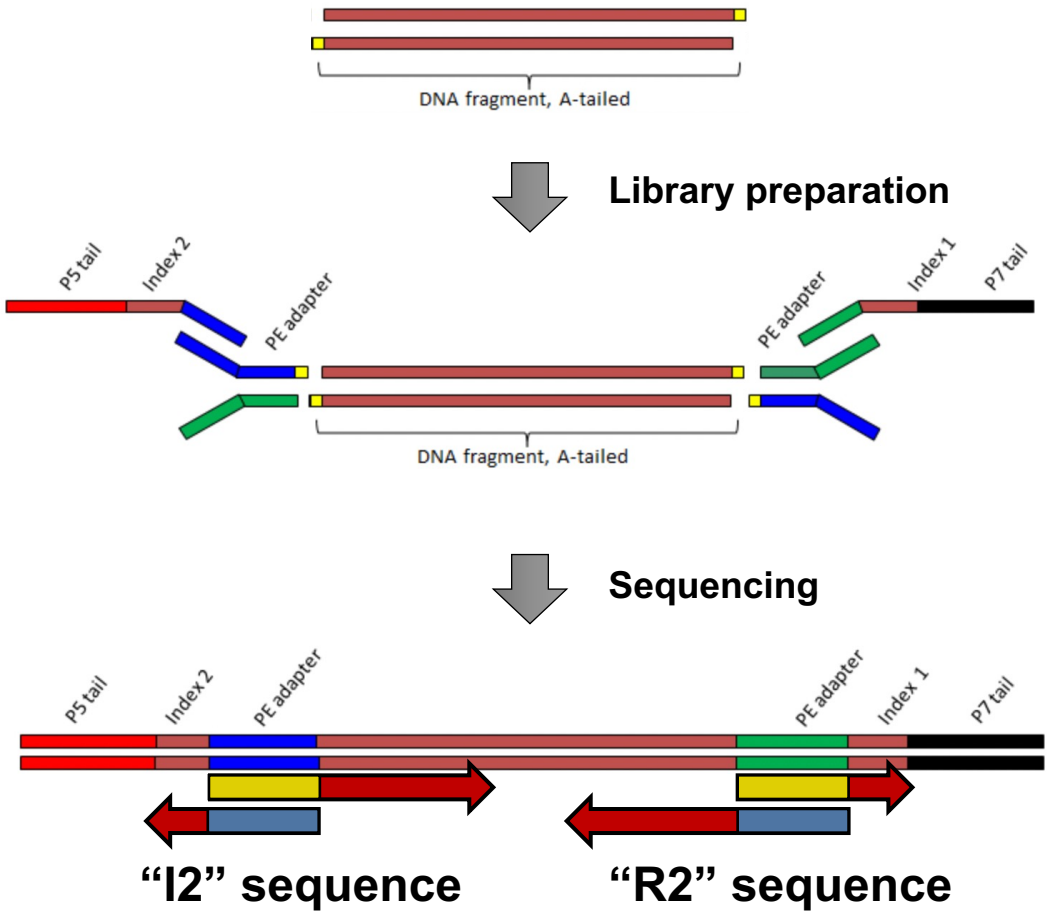
Why so many fastq files?



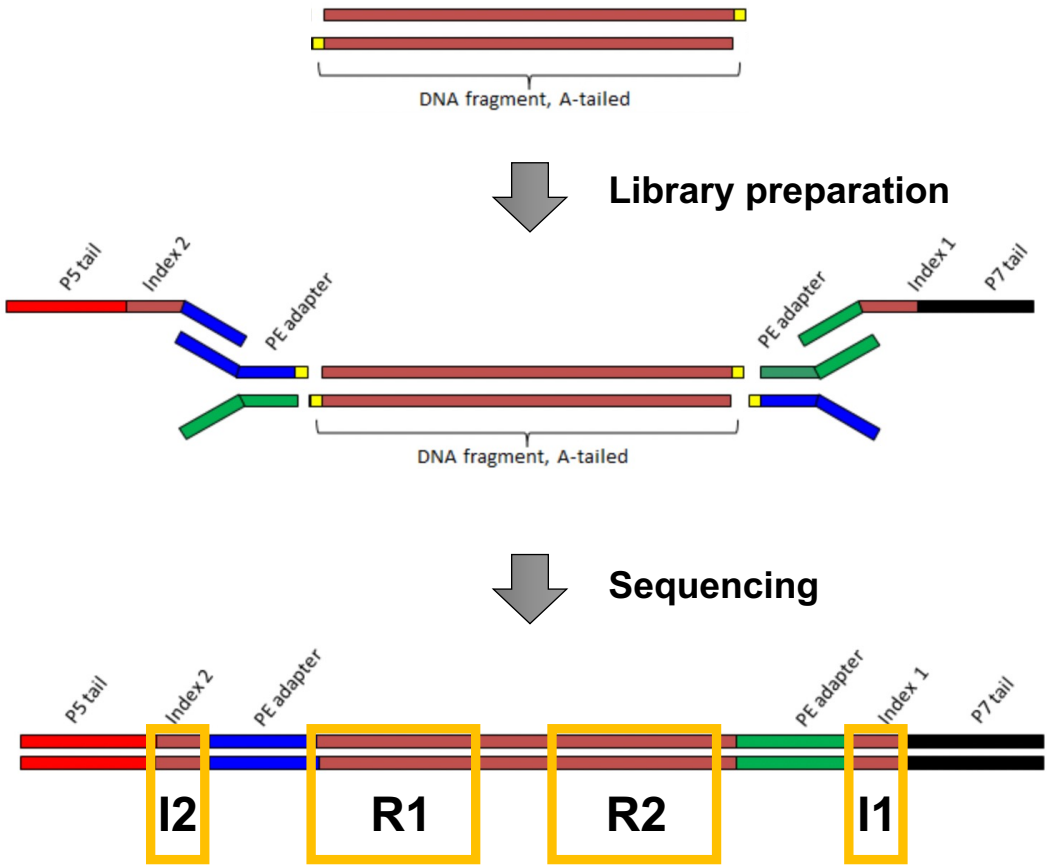
Why so many fastq files?



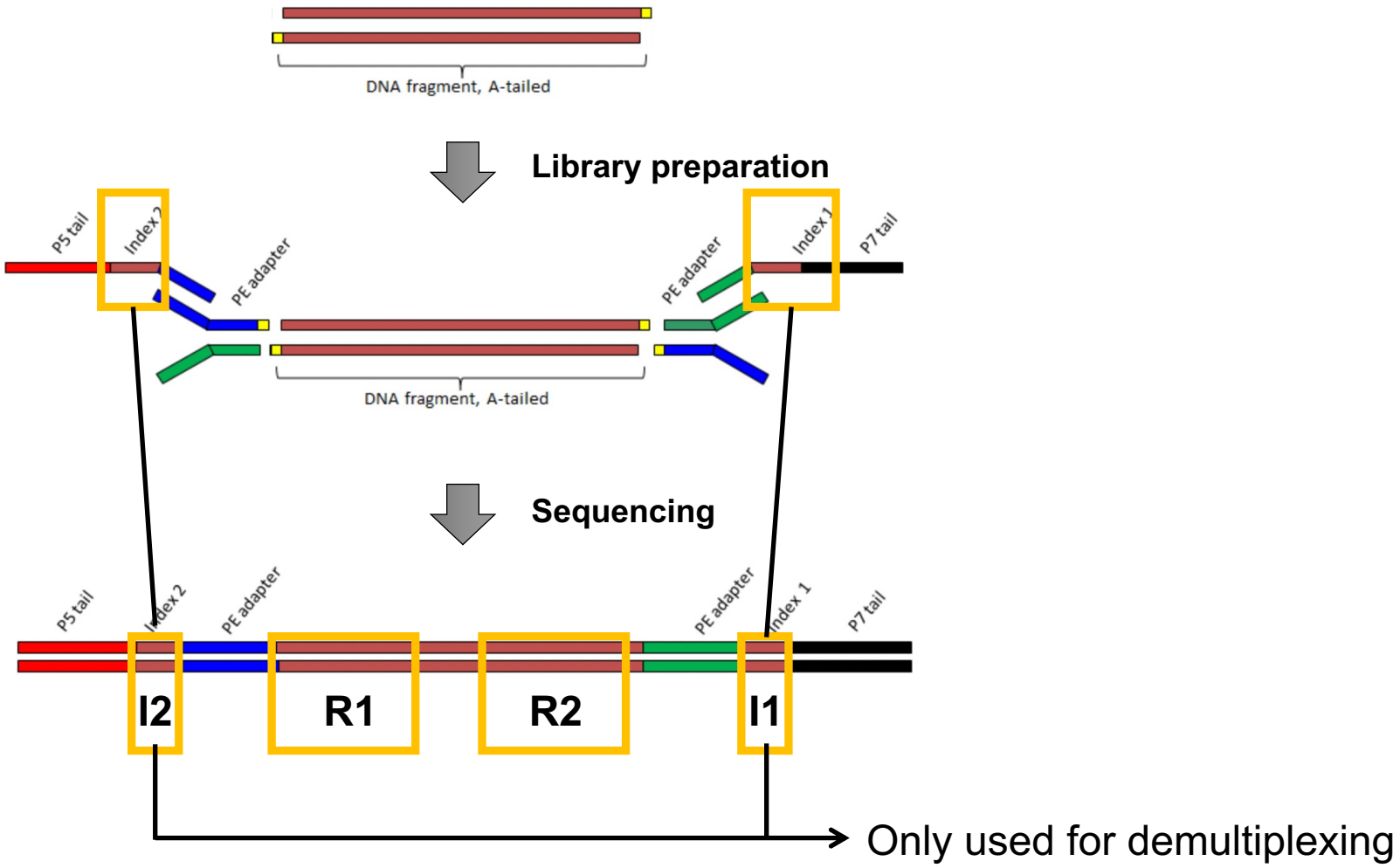
Why so many fastq files?



Why so many fastq files?



Why so many fastq files?



NGS processing workflow

- ✓ Get .bcl files 
- ✓ Create fastq files 

Or **`bcl2fastq`**

- QC: remove/trim low quality reads
- Align fastq to BAM
- Filter duplicates, artifacts, …
- Generate tracks
- Assay-specific downstream analysis

NGS processing workflow

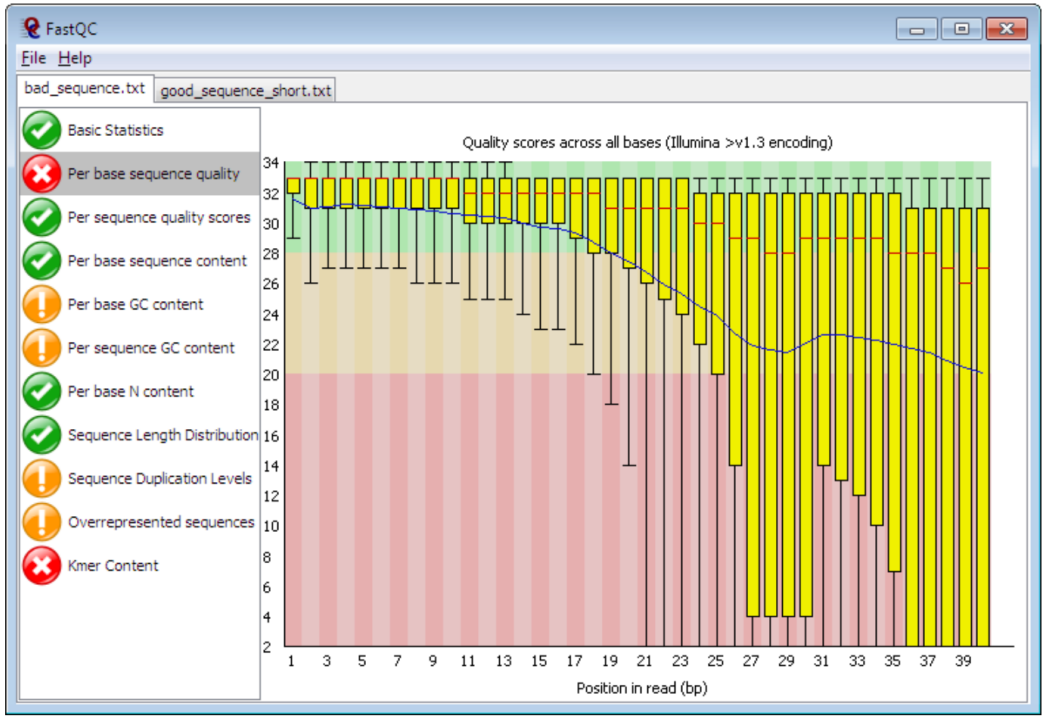
- Get .bcl files 
- Create fastq files  Or **`bcl2fastq`**

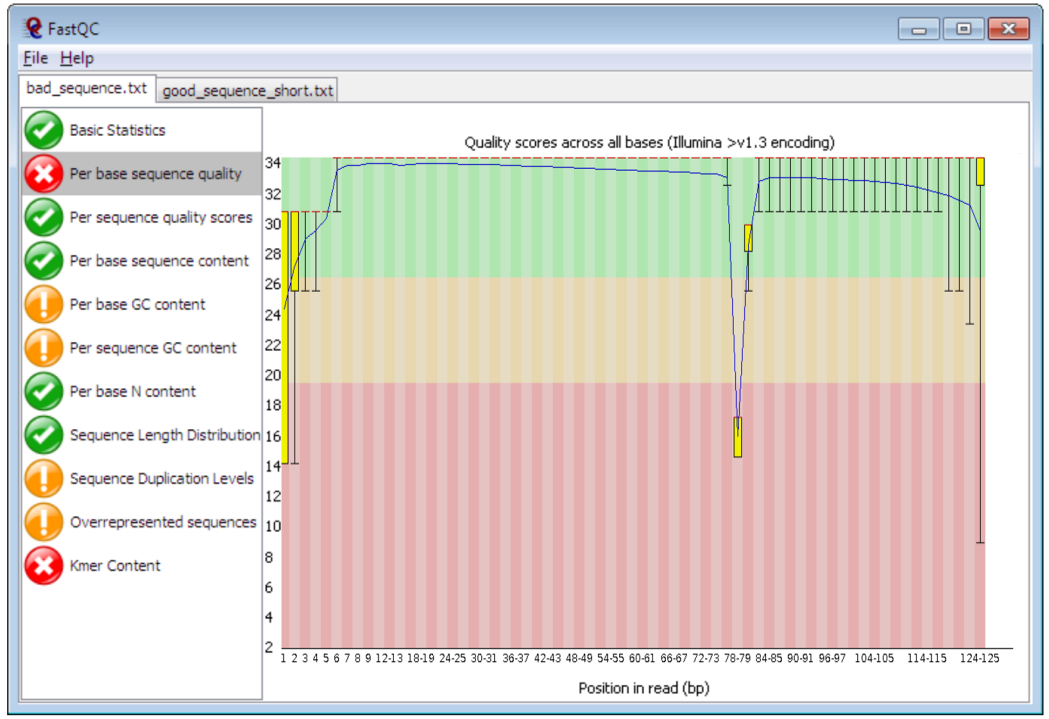
CHECK YOUR DATA

- QC: remove/trim low quality reads
- Align fastq to BAM
- Filter duplicates, artifacts, …
- Generate tracks
- Assay-specific downstream analysis

FastQC

FastQC is a program designed to spot potential problems in high througput sequencing datasets. It runs a set of analyses on one or more raw sequence files in fastq or bam format and produces a report which summarises the results.

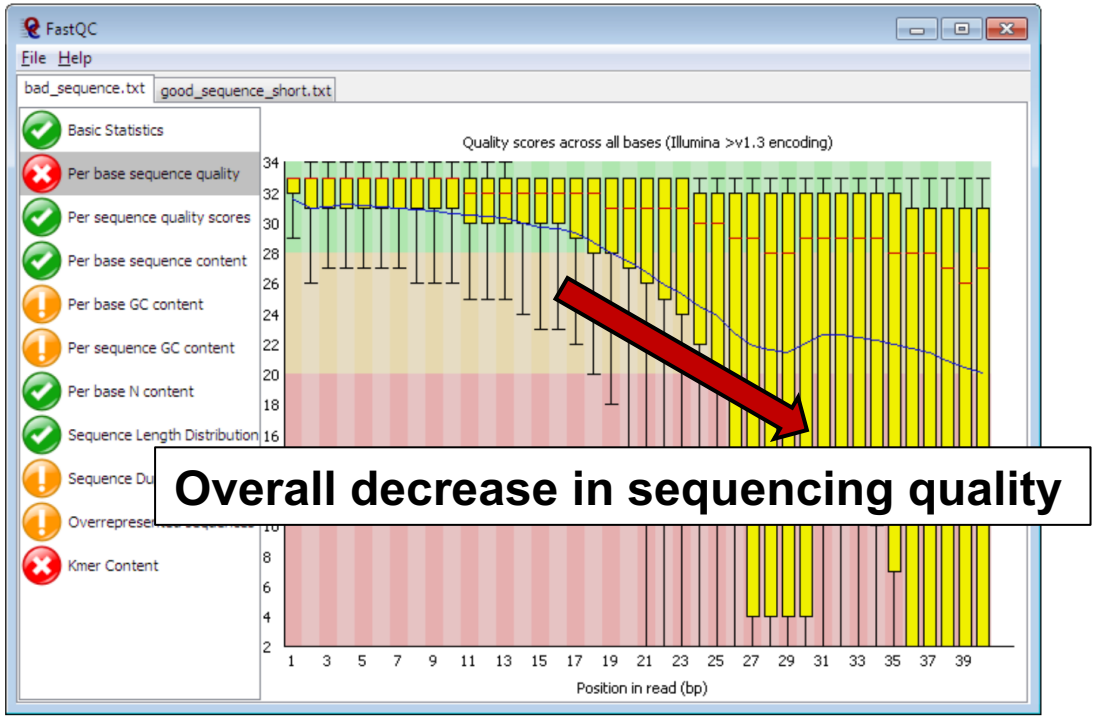


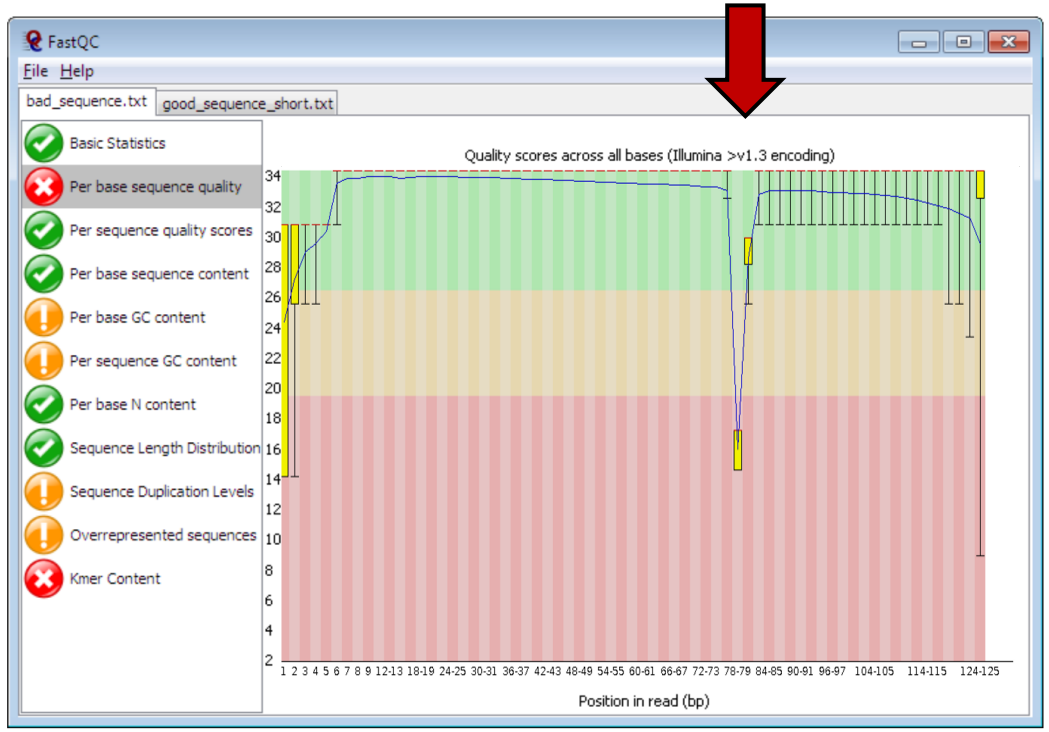


FastQC will highlight any areas where this library looks unusual and where you should take a closer look. The program is not tied to any specific type of sequencing technique and can be used to look at libraries coming from a large number of different experiment types (Genomic Sequencing, ChIP-Seq, RNA-Seq, BS-Seq etc etc).

FastQC

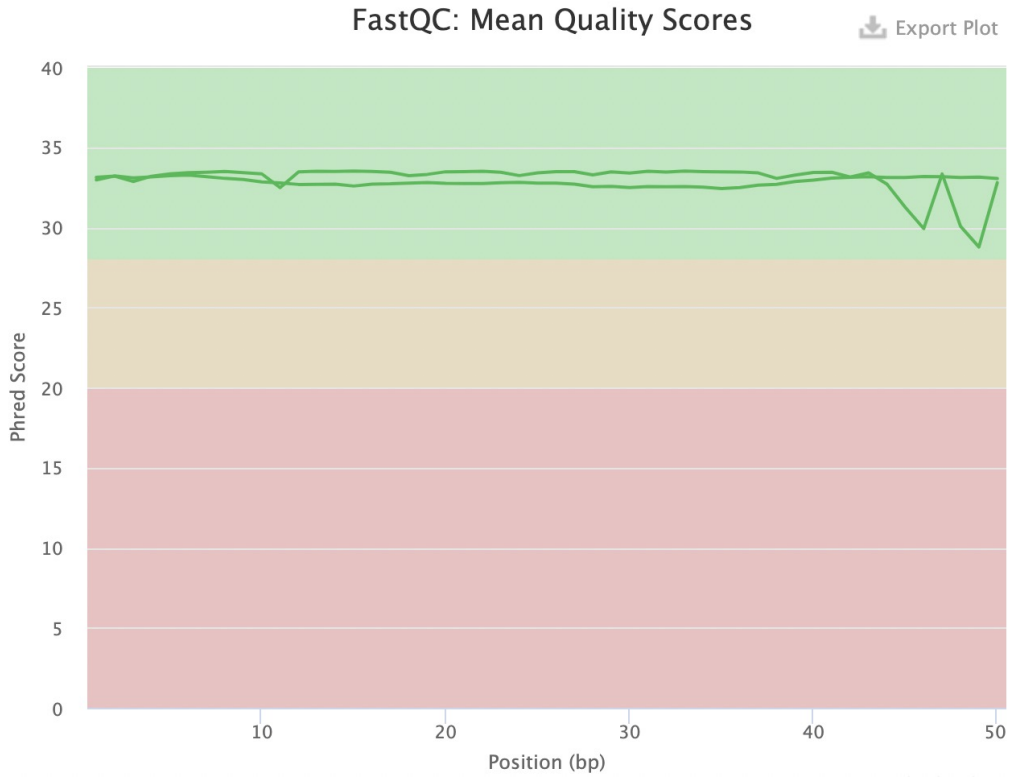
FastQC is a program designed to spot potential problems in high througput sequencing datasets. It runs a set of analyses on one or more raw sequence files in fastq or bam format and produces a report which summarises the results.





FastQC will highlight any areas where this library looks unusual and where you should take a closer look. The program is not tied to any specific type of sequencing technique and can be used to look at libraries coming from a large number of different experiment types (Genomic Sequencing, ChIP-Seq, RNA-Seq, BS-Seq etc etc).

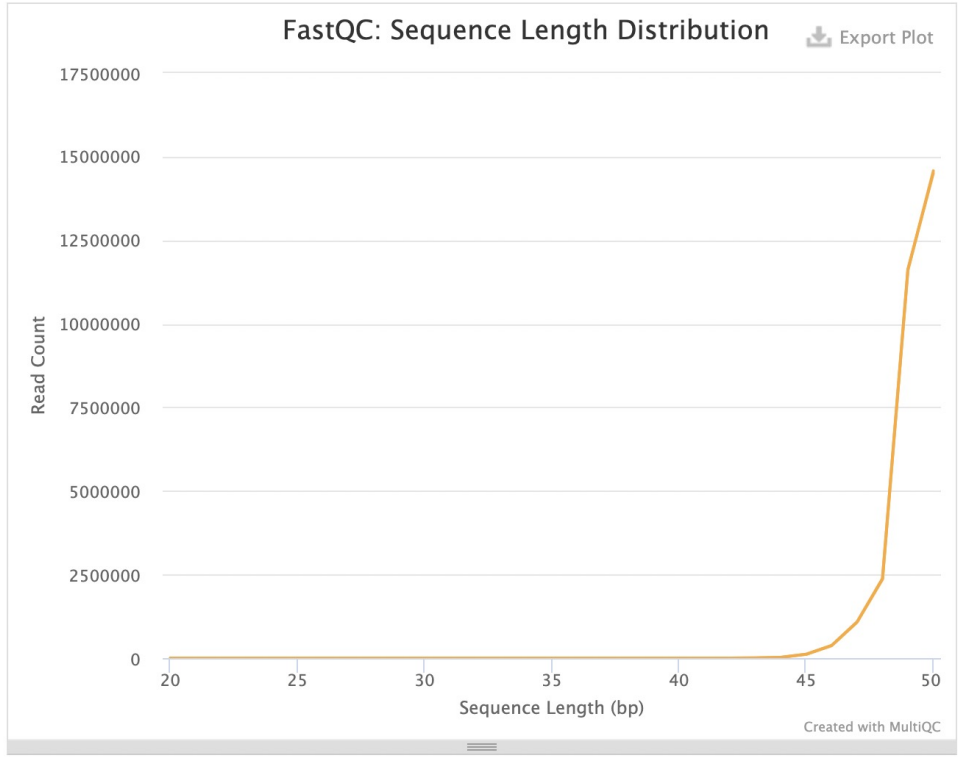
Cutadapt: trim away adapter sequences and low-quality ends



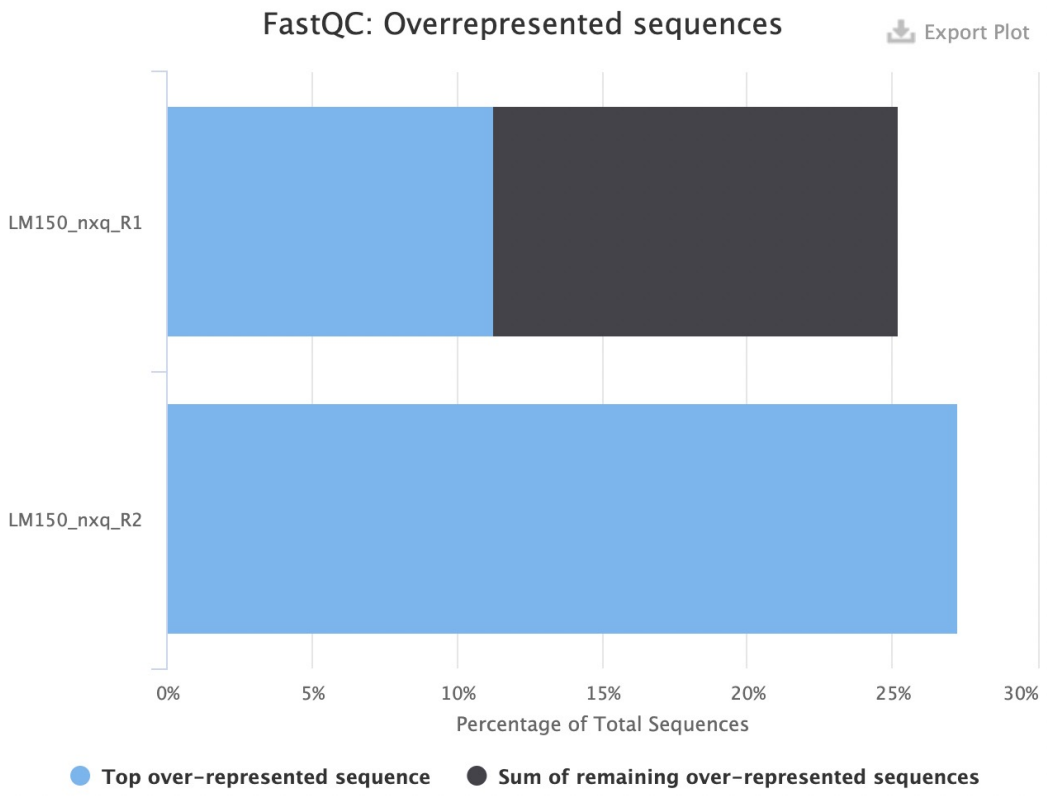
Cutadapt: trim away adapter sequences and low-quality ends

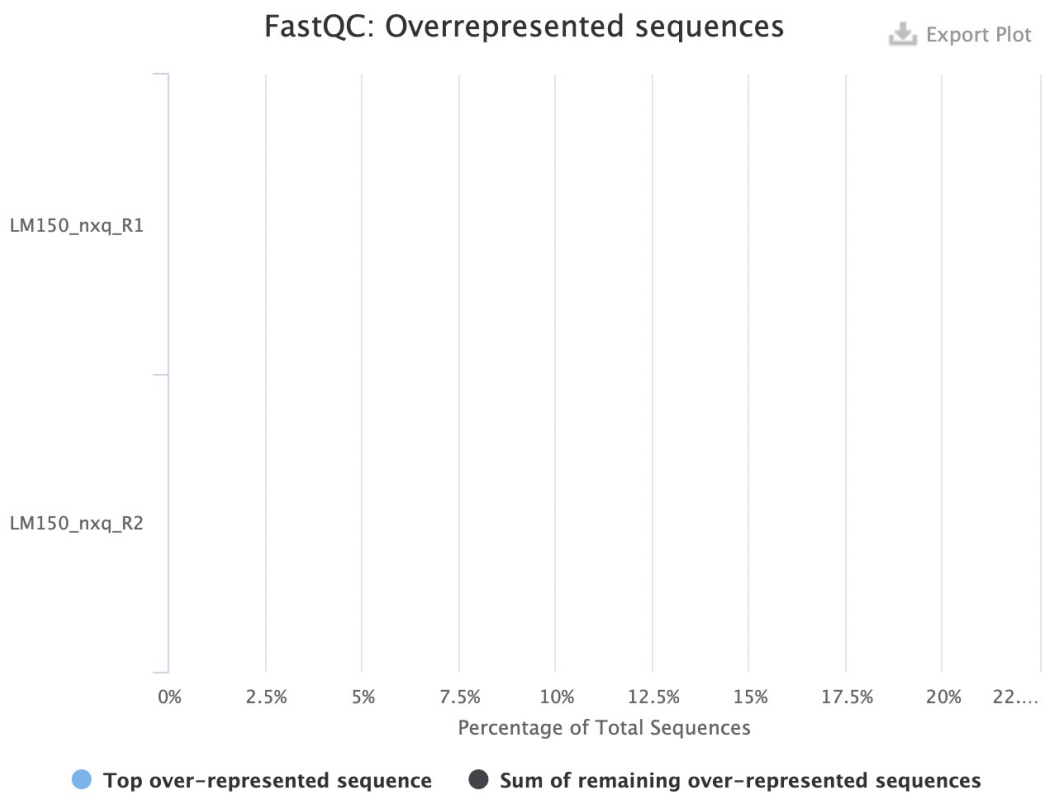
Sequence Length Distribution 80

All samples have sequences of a single length (50bp).



Cutadapt: trim away adapter sequences and low-quality ends

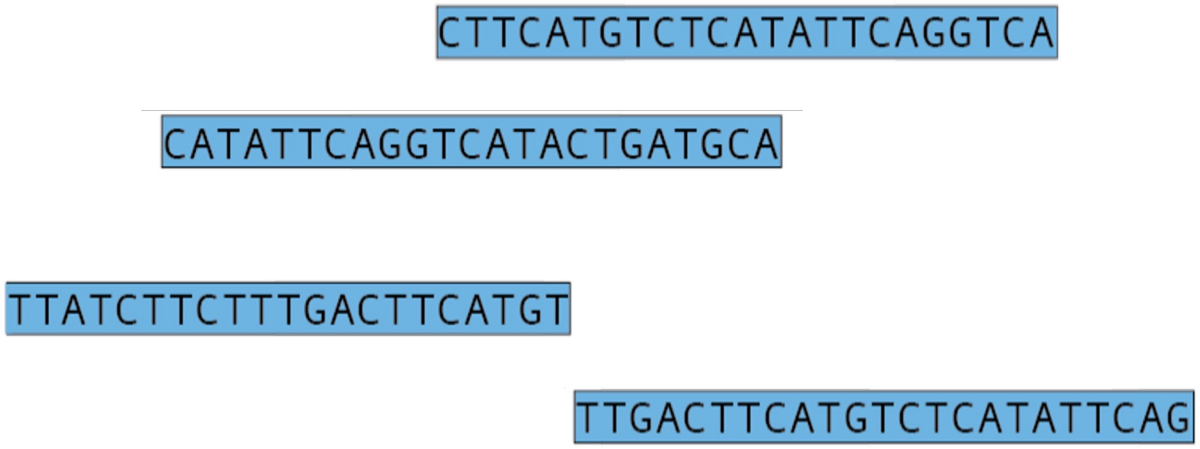




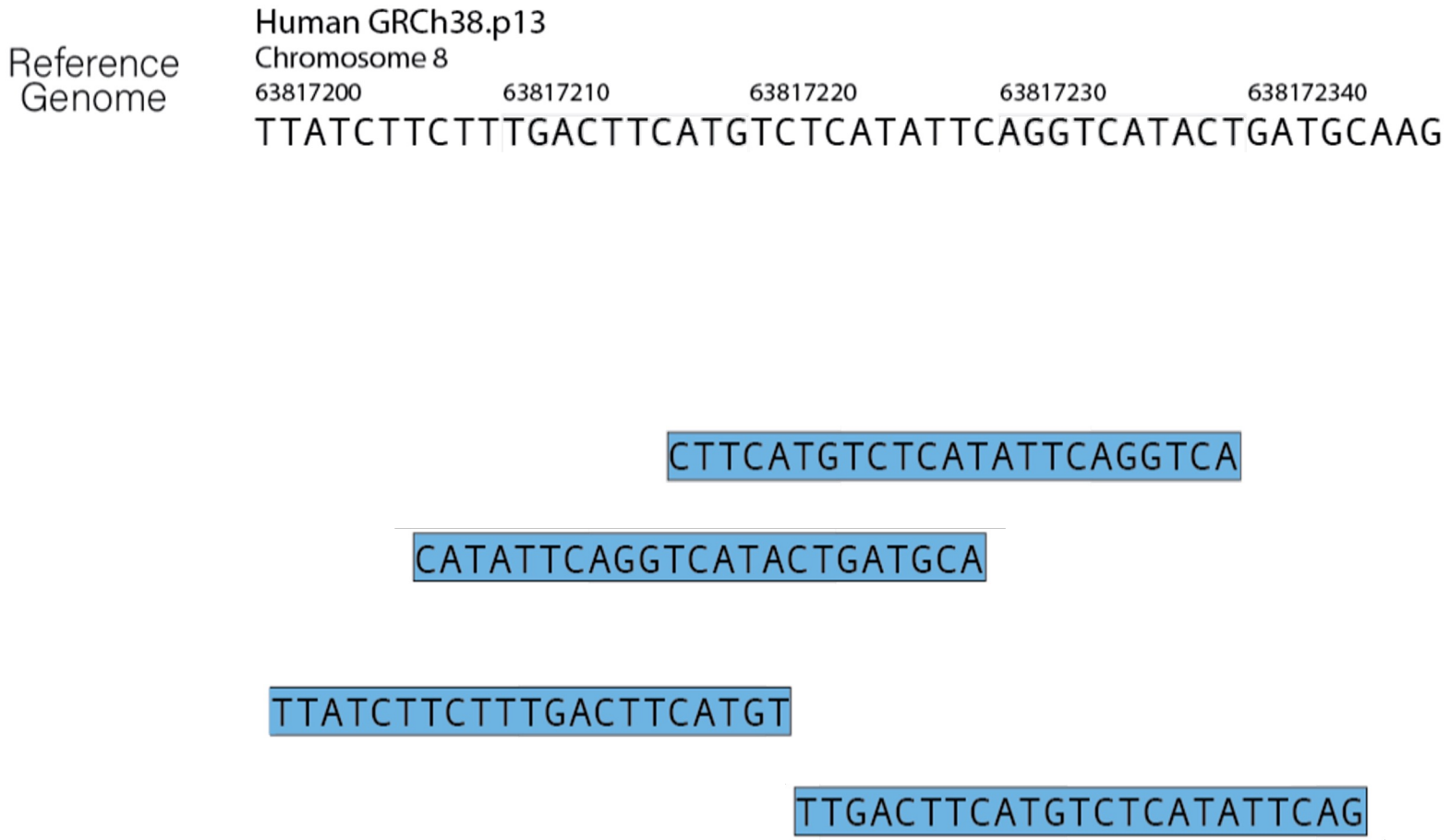
NGS processing workflow

- ✓ Get .bcl files 
- ✓ Create fastq files  Or **`bcl2fastq`**
- CHECK YOUR DATA E.g. **`FastQC`**
- ✓ QC: remove/trim low quality reads E.g. **`cutadapt`**
- ● Align fastq to BAM
- ● Filter duplicates, artifacts, …
- ● Generate tracks
- ● Assay-specific downstream analysis

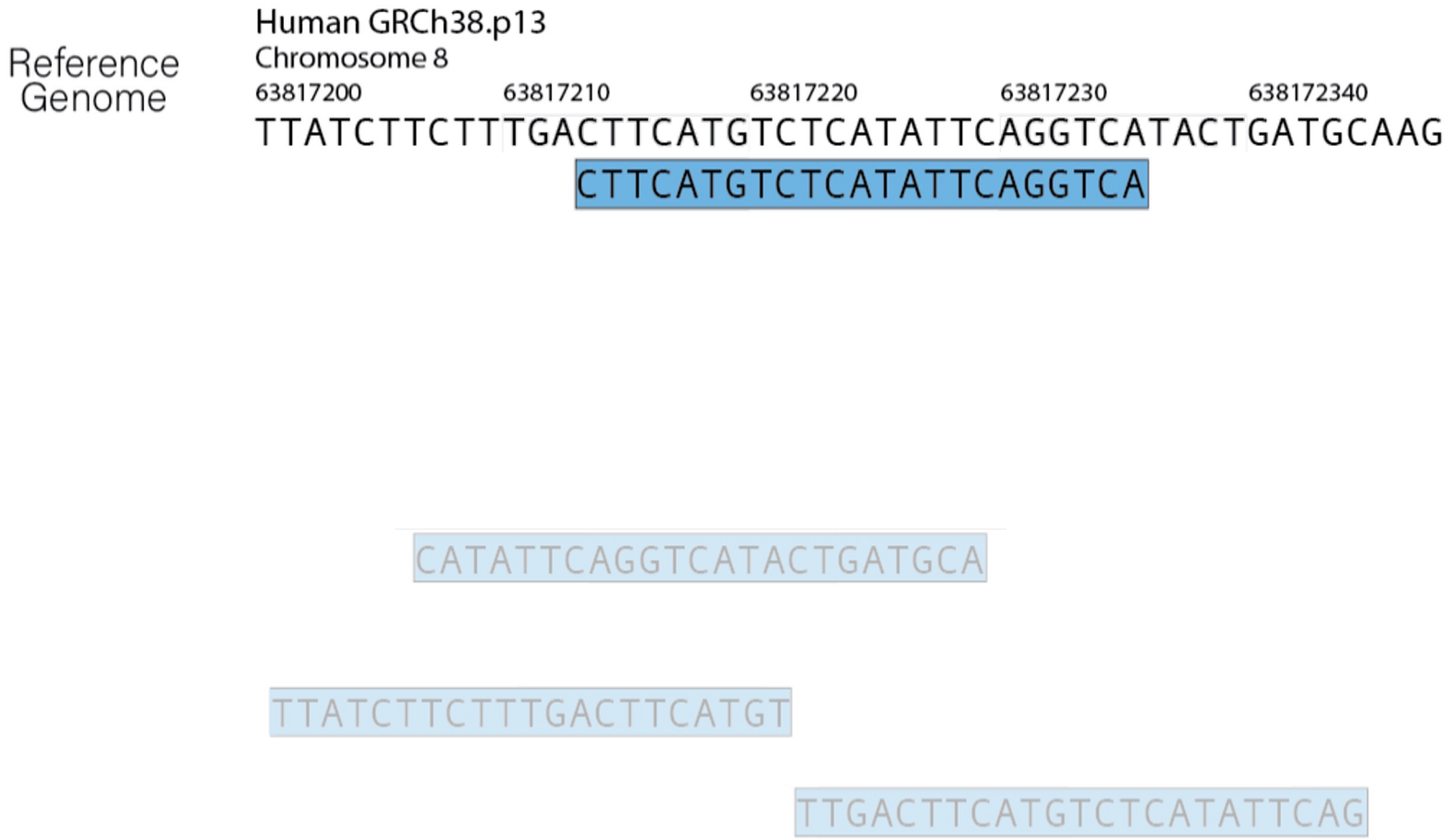
Mapping sequencing reads to a reference



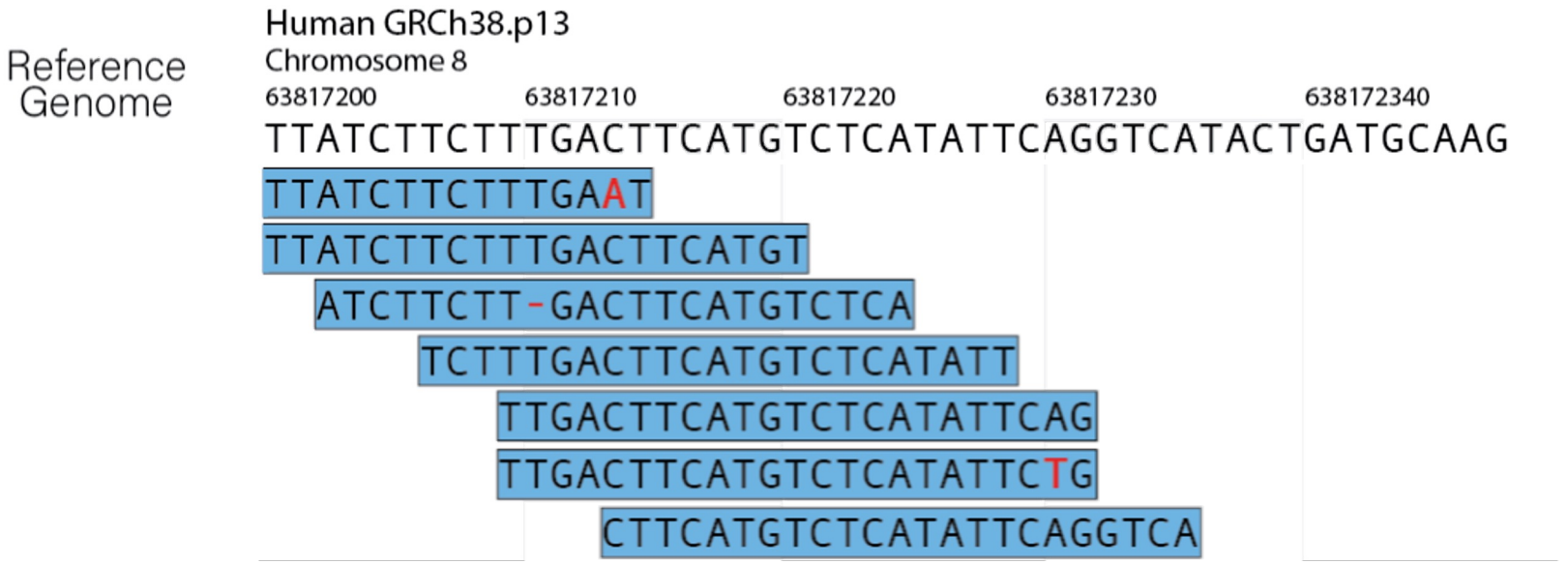
Mapping sequencing reads to a reference



Mapping sequencing reads to a reference



Mapping sequencing reads to a reference



SAM file format

Sequence Alignment Map (SAM) is a human-readable, rectangular, text-based format for storing biological sequences aligned to a reference sequence.

Each entry (line) describes where a read is mapped on the reference and how it is mapped

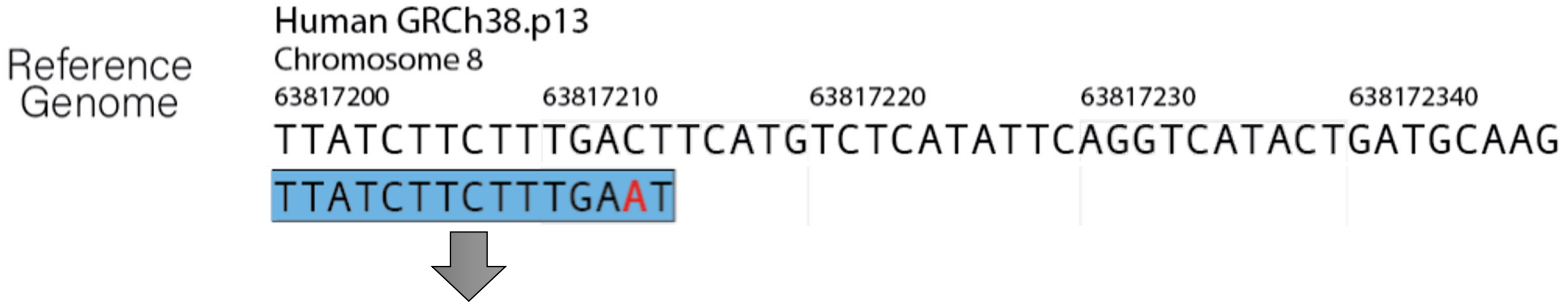
Col	Field	Type	Brief description
1	QNAME	String	Query template NAME
2	FLAG	Int	bitwise FLAG
3	RNAME	String	References sequence NAME
4	POS	Int	1- based leftmost mapping POSition
5	MAPQ	Int	MAPping Quality
6	CIGAR	String	CIGAR string
7	RNEXT	String	Ref. name of the mate/next read
8	PNEXT	Int	Position of the mate/next read
9	TLEN	Int	observed Template LENgth
10	SEQ	String	segment SEQuence
11	QUAL	String	ASCII of Phred-scaled base QUALity+33

SAM file format

Sequence Alignment Map (SAM) is a human-readable, rectangular, text-based format for storing biological sequences aligned to a reference sequence.

Each entry (line) describes where a read is mapped on the reference and how it is mapped

Col	Field	Type	Brief description
1	QNAME	String	Query template NAME
2	FLAG	Int	bitwise FLAG
3	RNAME	String	References sequence NAME
4	POS	Int	1- based leftmost mapping POSition
5	MAPQ	Int	MAPping Quality
6	CIGAR	String	CIGAR string
7	RNEXT	String	Ref. name of the mate/next read
8	PNEXT	Int	Position of the mate/next read
9	TLEN	Int	observed Template LENgth
10	SEQ	String	segment SEQuence
11	QUAL	String	ASCII of Phred-scaled base QUALity+33



Read name	Flag	Chr	Position	Length	CIGAR	Read name (mate)	Chr (mate)	Position (mate)	Sequence	Per base sequencing quality
HWI-ST330:304:H045HADXX:2093#1	2	chr8	63817200	50	14M1X	2093#2	chr8	6381932	TTATCTTCTTTGAAT	?????BBBBBBDD=?

BAM file format

BAM files are **binarized** SAM files, allowing great compression of the alignment results.

However, bam files are not directly human-readable.

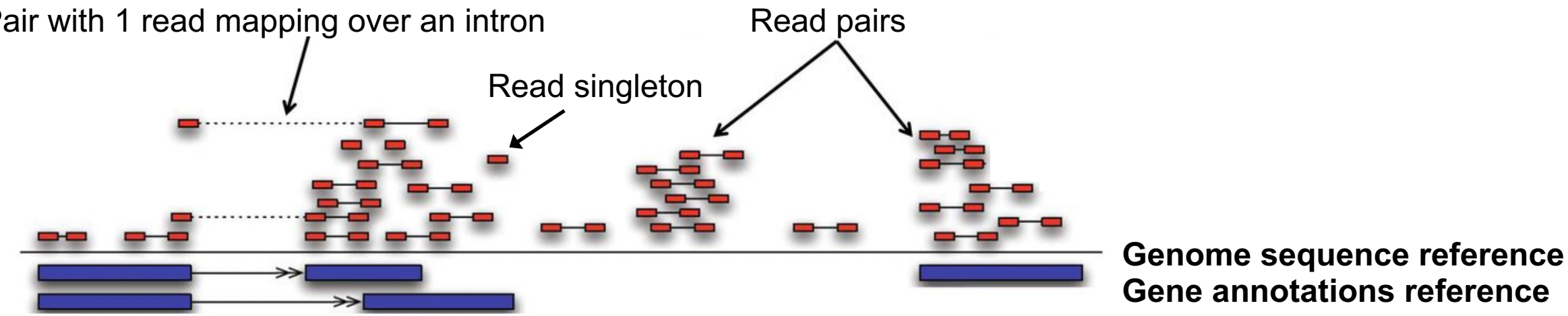
```
samtools view --bam ….sam > ….bam
```

Mapping tools

There are a plethora of alignment tools.

Each one requires the genome reference to be indexed first.

Some mappers can be ”**splice-aware**”, allowing reads to be mapped over annotated introns.



NGS processing workflow

- Get .bcl files 
- Create fastq files  Or **`bcl2fastq`**
- QC: remove/trim low quality reads E.g. **`cutadapt`**
- Align fastq to BAM E.g. **`bowtie2`**
- Filter duplicates, artifacts, …
- Generate tracks
- Assay-specific downstream analysis

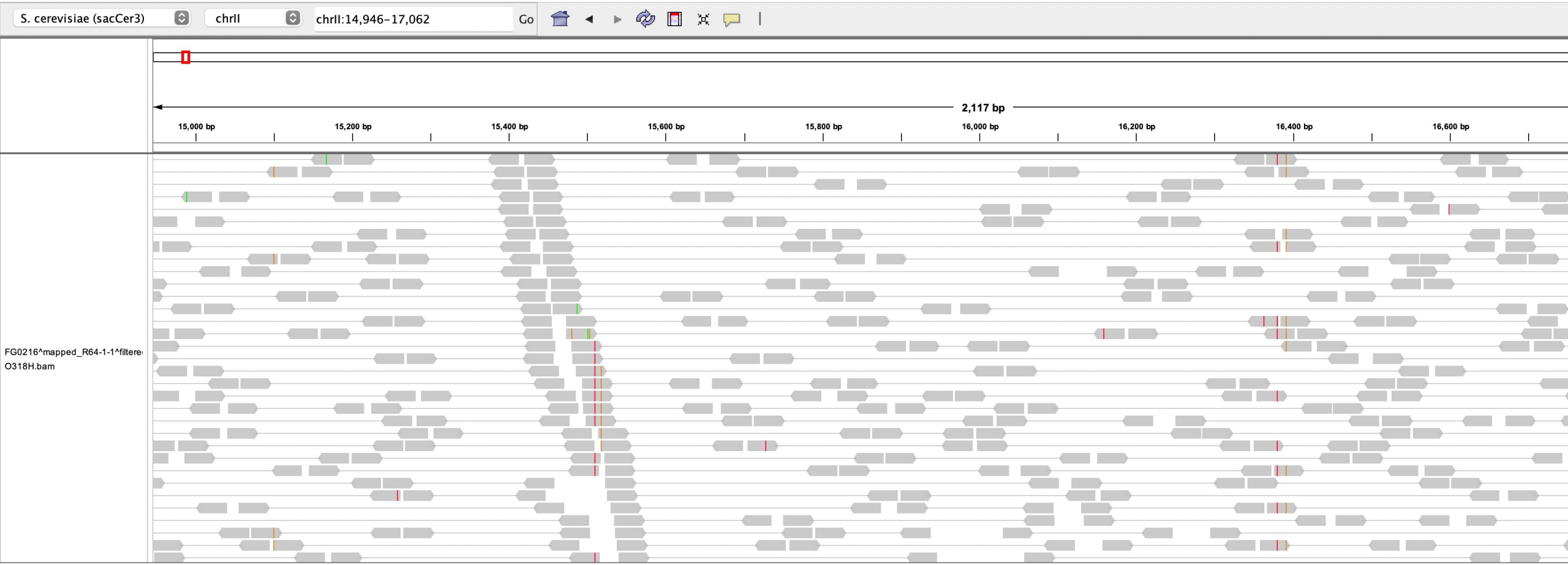
NGS processing workflow

- Get .bcl files 
- Create fastq files  Or **`bcl2fastq`**
- QC: remove/trim low quality reads E.g. **`cutadapt`**
- Align fastq to BAM E.g. **`bowtie2`**

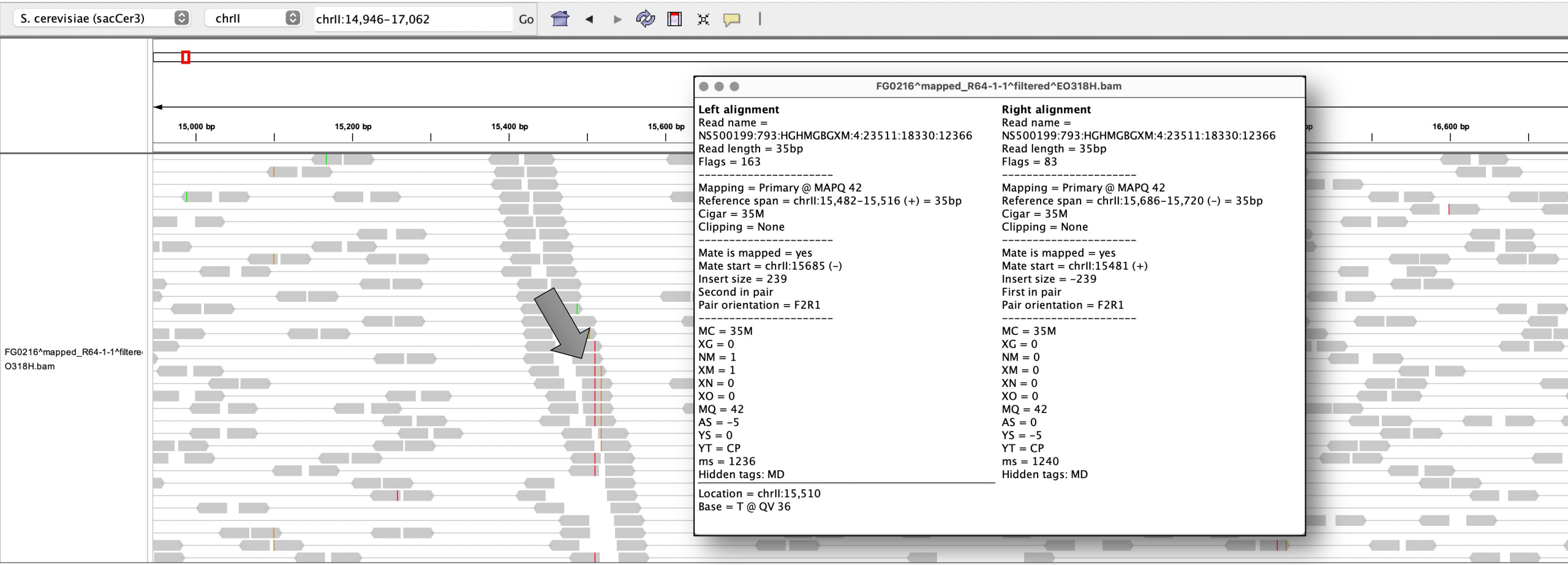
CHECK YOUR DATA

- Filter duplicates, artifacts, …
- Generate tracks
- Assay-specific downstream analysis

IGV: Integrative Genome Browser

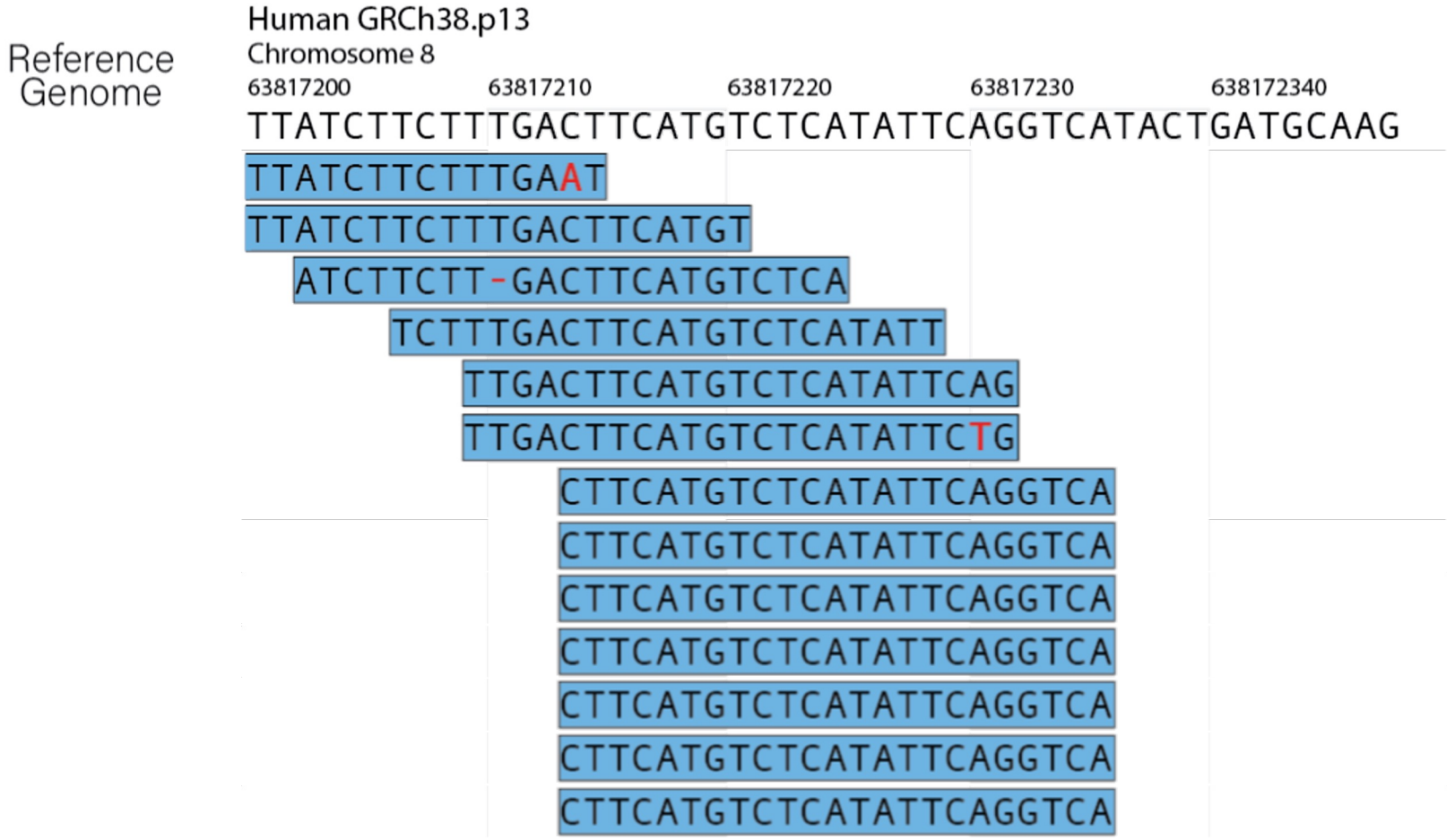


IGV: Integrative Genome Browser



Filtering duplicates

Multiple reads (fragments) with same mapping position (start & end) can be viewed as PCR duplicates.



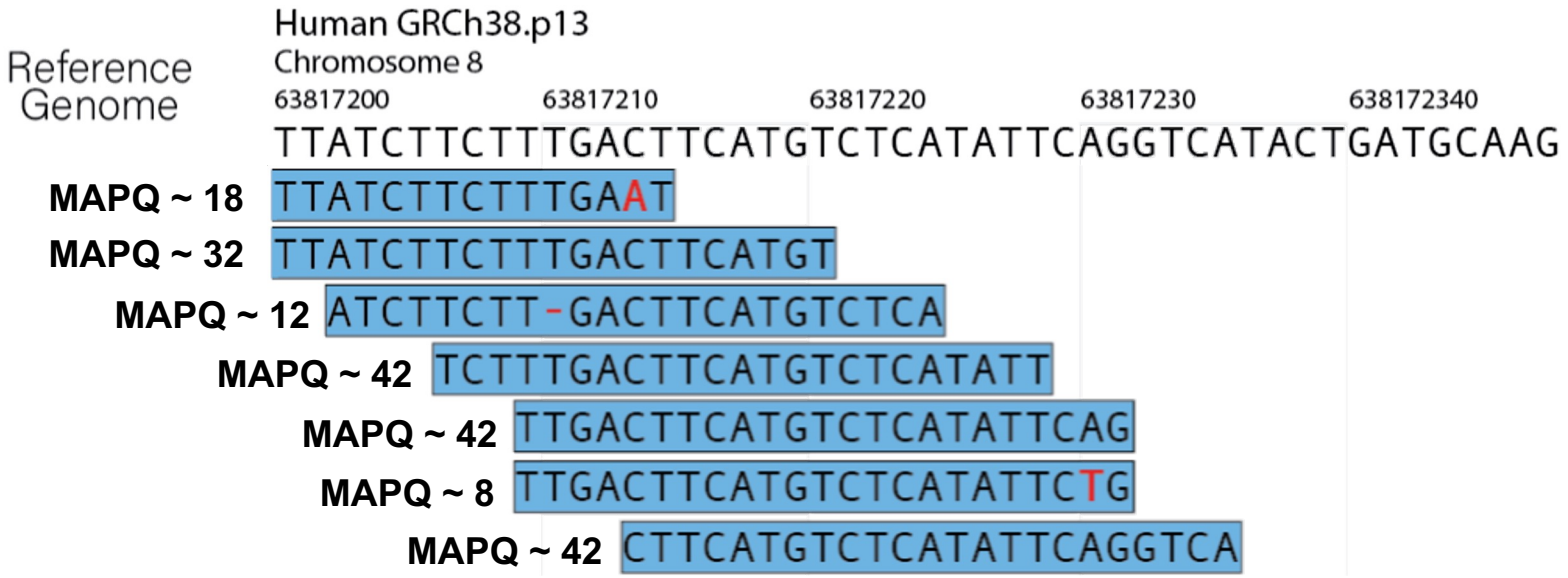
Filtering duplicates

Multiple reads (fragments) with same mapping position (start & end) can be viewed as PCR duplicates.

Filtering out low-quality mapping reads

Mapping Quality Scores (**MAPQ**) quantify the probability that a read is misplaced.

$$MAPQ = -10 * log_{10}\big(P(read\ is\ wrongly\ mapped)\big)$$

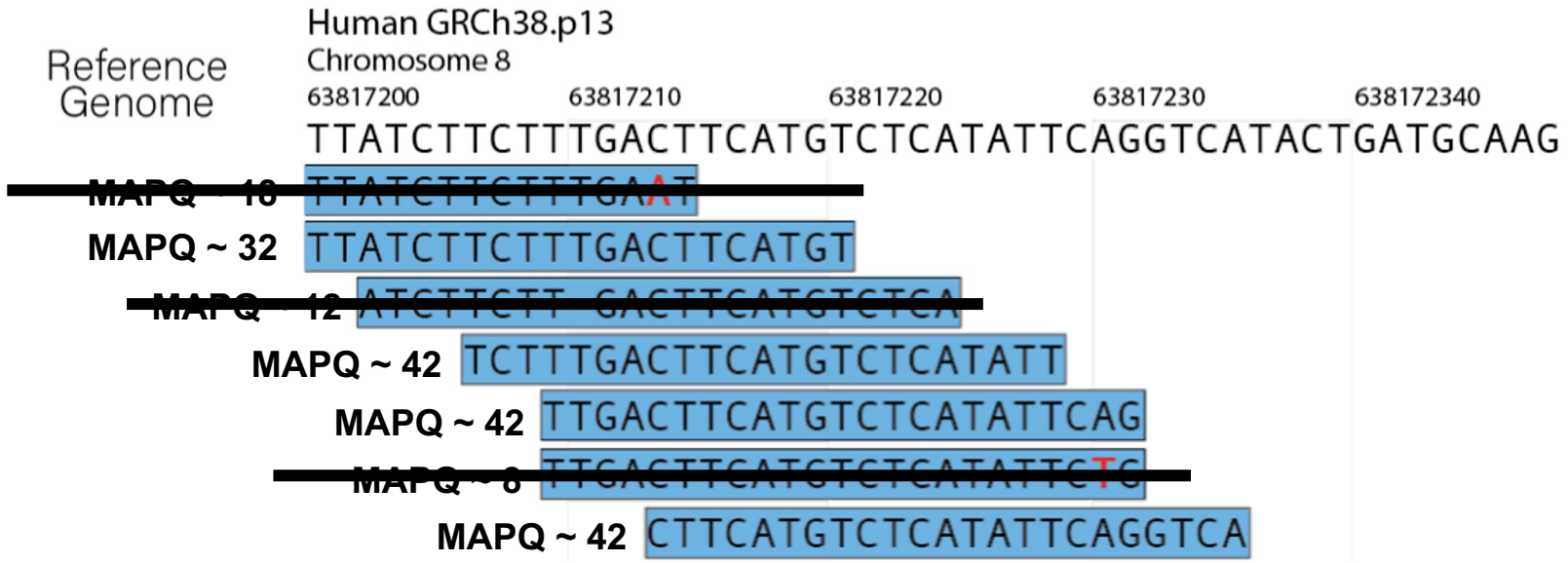


Filtering out low-quality mapping reads

Mapping Quality Scores (**MAPQ**) quantify the probability that a read is misplaced.

$$MAPQ = -10 * log_{10}\big(P(read\ is\ wrongly\ mapped)\big)$$

For example, a MAPQ score of 20 indicates that the probability for the read to be map at the indicated position is 0.01.

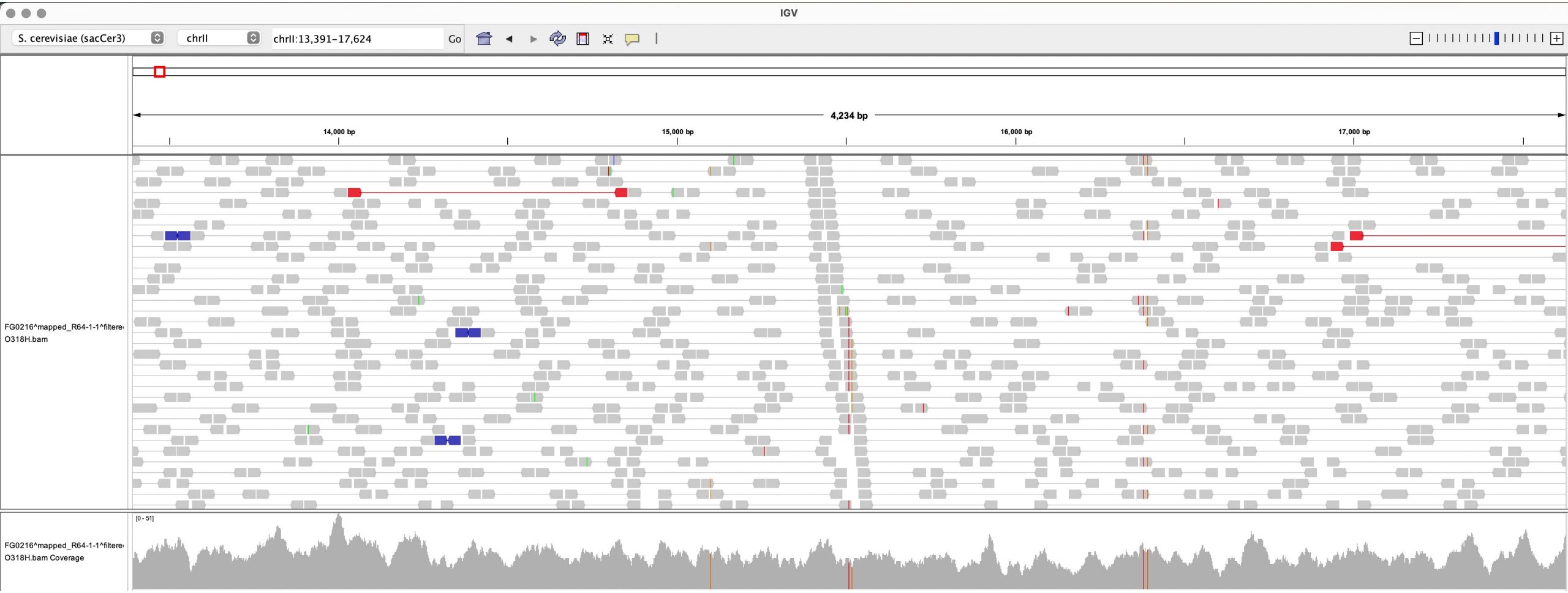


NGS processing workflow

- ✓ Get .bcl files 
- ✓ Create fastq files  Or **`bcl2fastq`**
- ✓ QC: remove/trim low quality reads E.g. **`cutadapt`**
- ✓ Align fastq to BAM E.g. **`bowtie2`**
- ✓ Filter duplicates, artifacts, … E.g. **`samtools`**
- ● Generate tracks
- ● Assay-specific downstream analysis

Generating tracks from mapped reads

Basic approach: ”pile-up” of all the fragments in `bam` files to generate a **coverage track**.



deepTools: a software suite to manage/produce genomic tracks

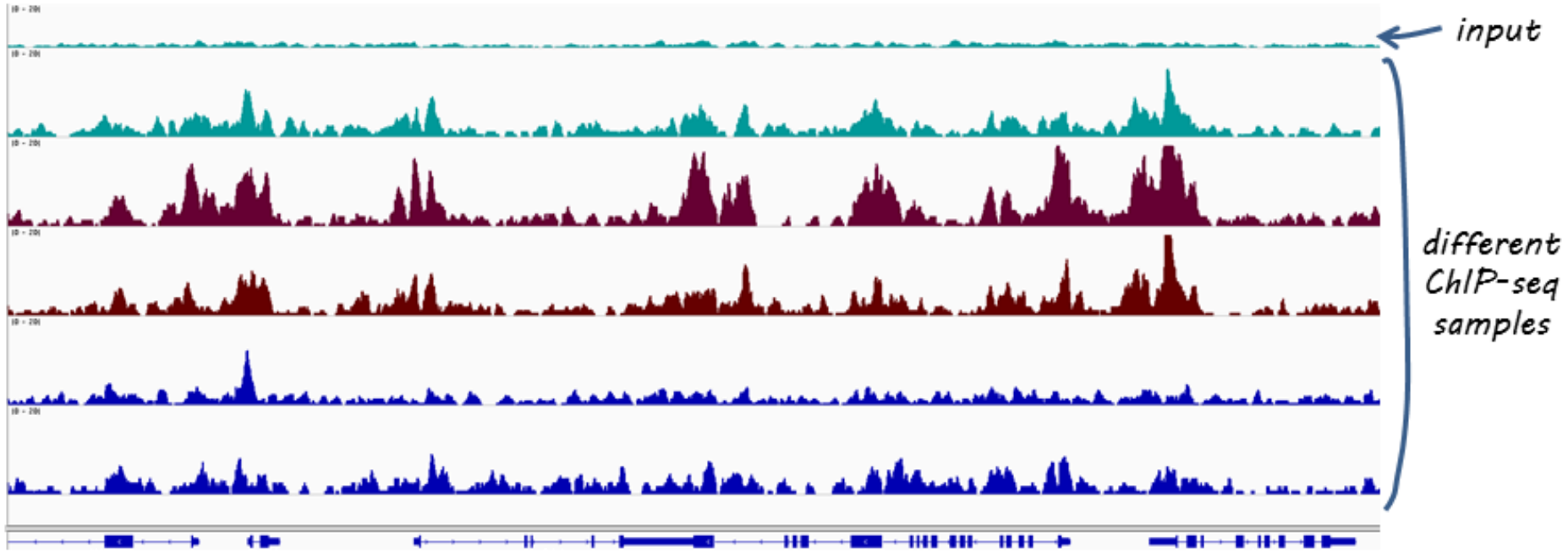
https://deeptools.readthedocs.io/en/latest/content/list_of_tools.html

- Tools for BAM and bigWig file processing
  - multiBamSummary
  - multiBigwigSummary
  - correctGCBias
  - bamCoverage
  - bamCompare
  - bigwigCompare
  - bigwigAverage
  - computeMatrix
  - alignmentSieve

- Tools for QC
  - plotCorrelation
  - plotPCA
  - plotFingerprint
  - bamPEFragmentSize
  - computeGCBias
  - plotCoverage
- Heatmaps and summary plots
  - plotHeatmap
  - plotProfile
  - plotEnrichment
- Miscellaneous
  - computeMatrixOperations
  - estimateReadFiltering

deepTools: a software suite to manage/produce genomic tracks

for visualizing continuous data, e·g· in the UCSC Genome Browser or IGV, bigWig files come in really handy



remember that there are 2 deepTools for bam → bigWig conversion:

- ***bamCoverage:*** *for individual files (like those shown here)*
- ***bamCompare:*** *to normalize two files to each other*

NGS processing workflow

- Get .bcl files 
- Create fastq files  Or **`bcl2fastq`**
- QC: remove/trim low quality reads — E.g. **`cutadapt`**
- Align fastq to BAM — E.g. **`bowtie2`**
- Filter duplicates, artifacts, … — E.g. **`samtools`**
- Generate tracks — E.g. **`deepTools`**
- Assay-specific downstream analysis

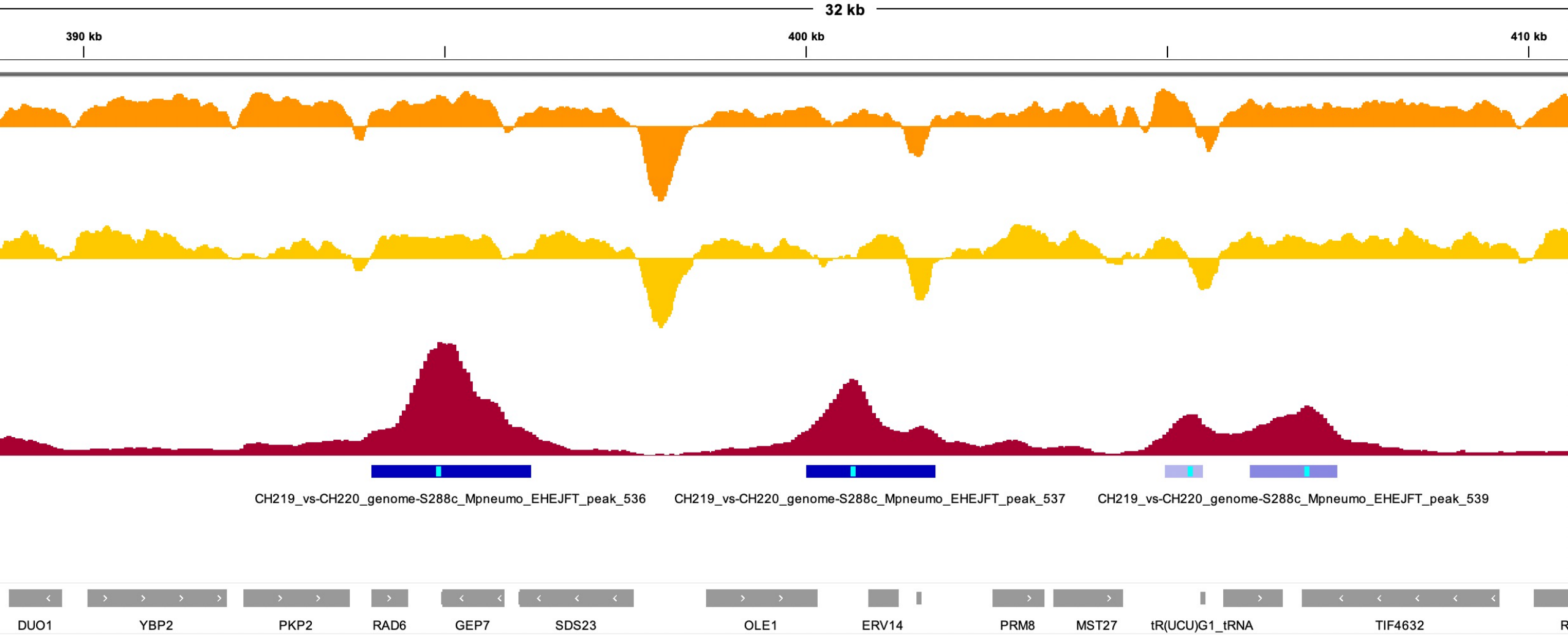
NGS processing workflow

- Get .bcl files 
- Create fastq files  Or **`bcl2fastq`**
- QC: remove/trim low quality reads — E.g. **`cutadapt`**
- Align fastq to BAM — E.g. **`bowtie2`**
- Filter duplicates, artifacts, … — E.g. **`samtools`**
- Generate tracks — E.g. **`deepTools`**

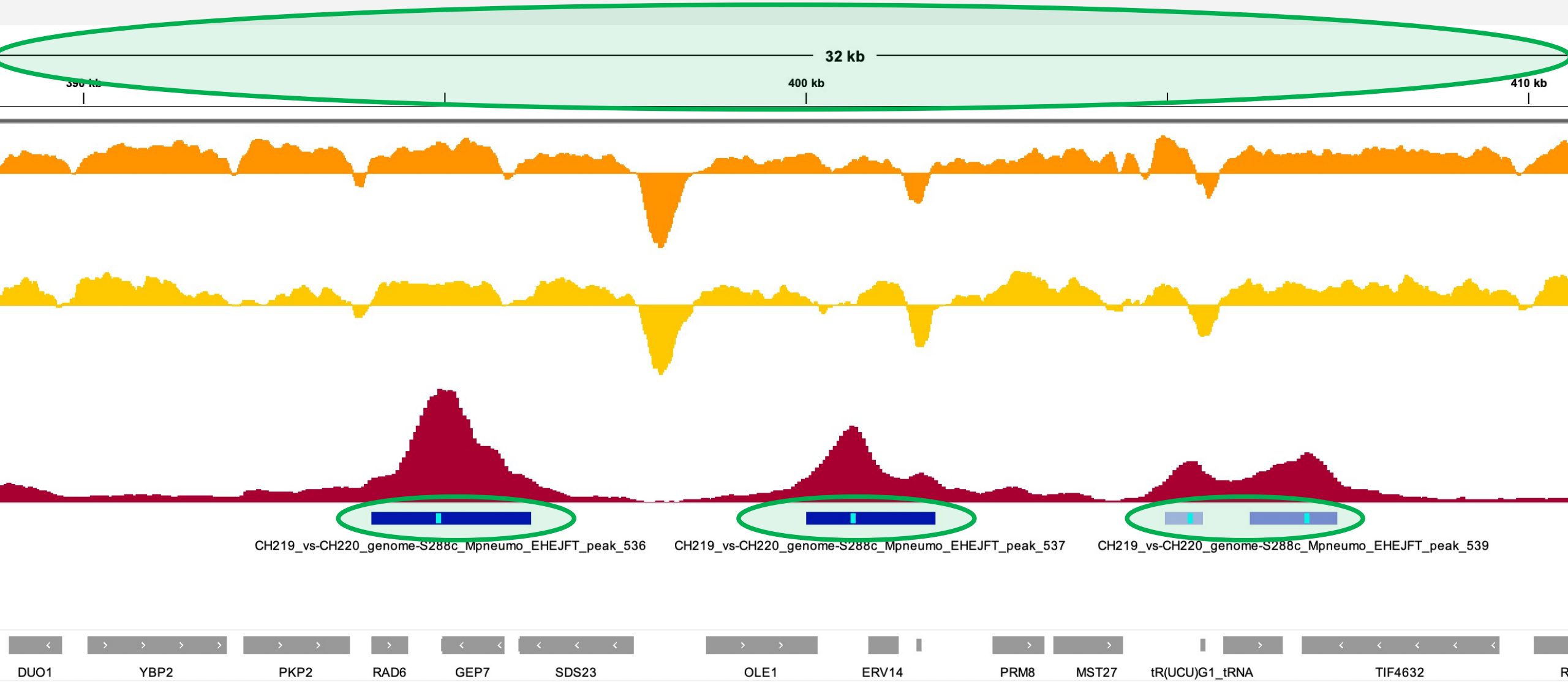
CHECK YOUR DATA

- Assay-specific downstream analysis

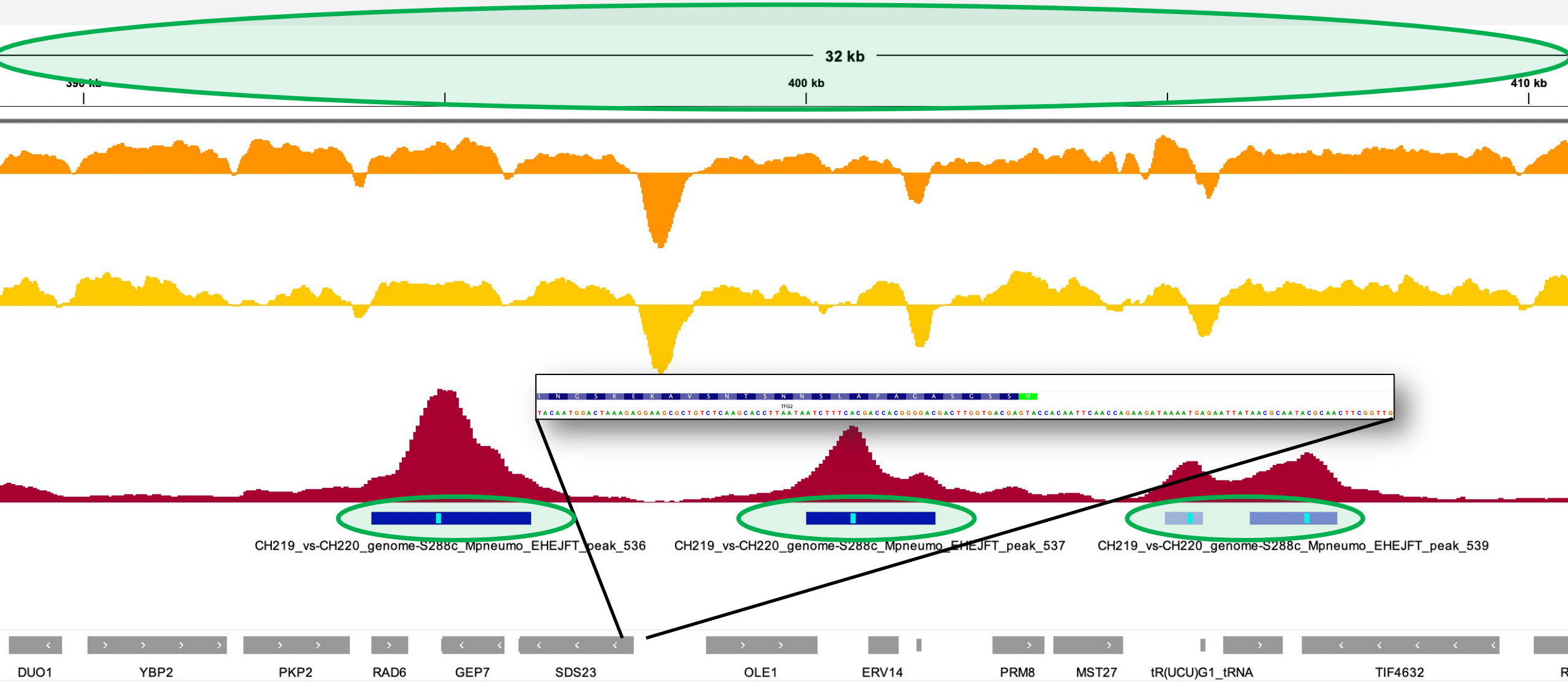
Epigenomics in a browser



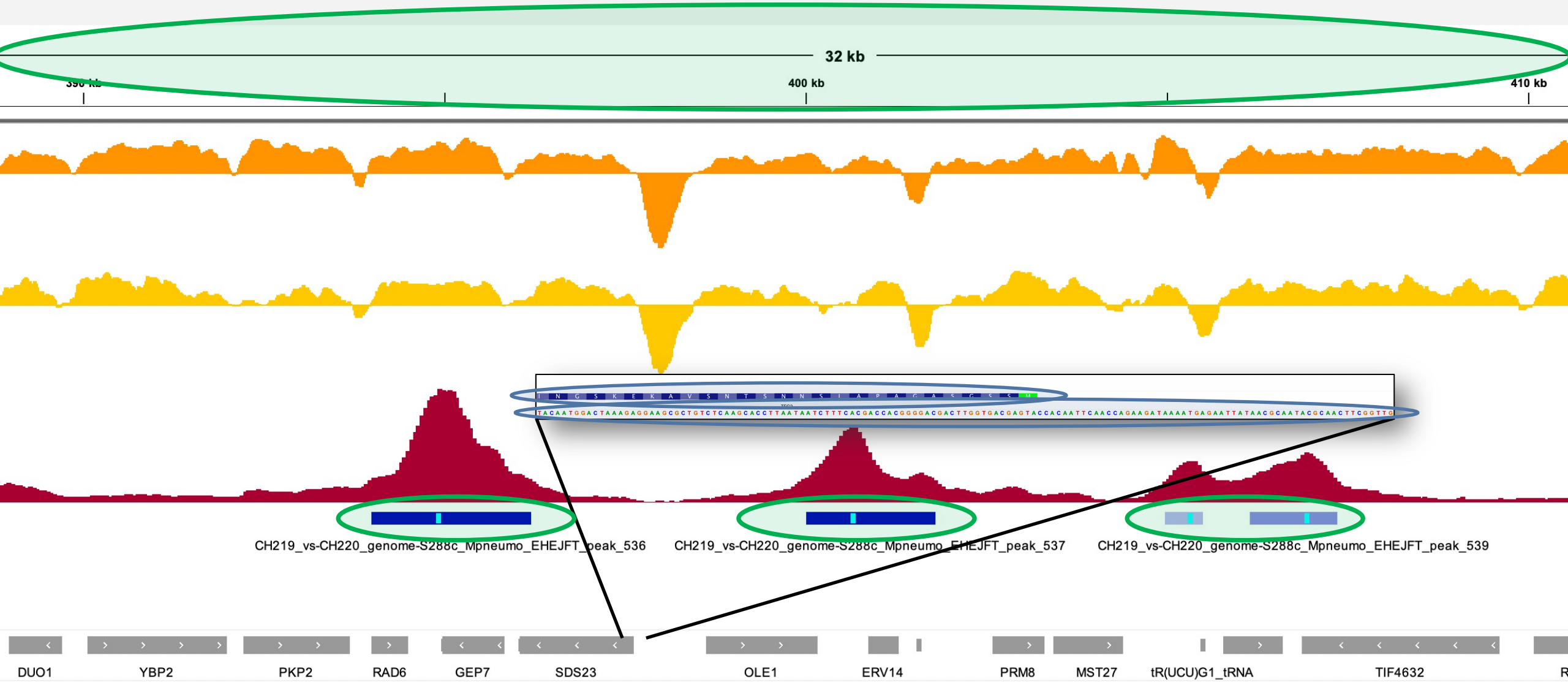
Epigenomics in a browser



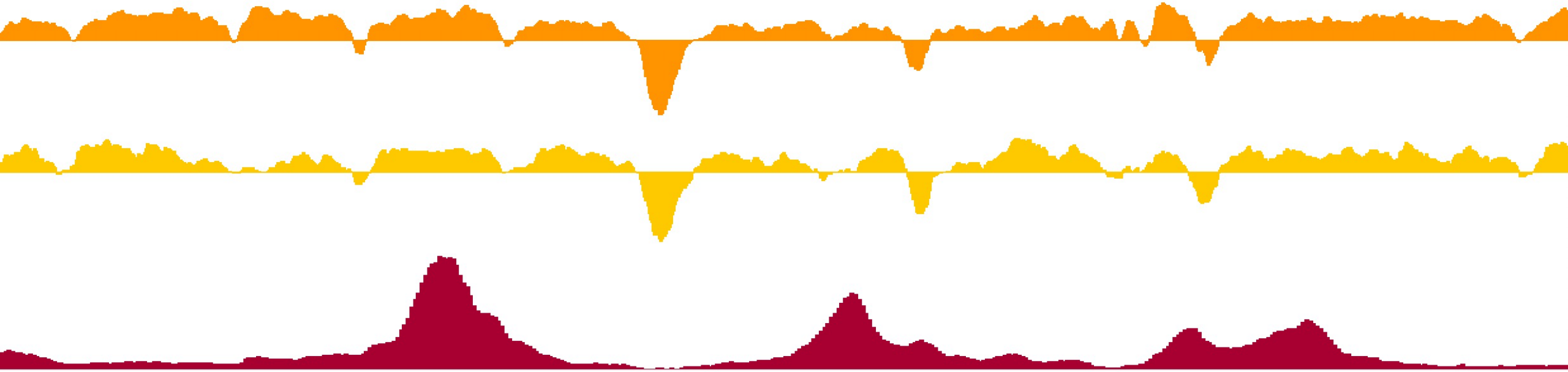
Epigenomics in a browser



Epigenomics in a browser



Epigenomics in a browser



Epigenomics in a browser

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format

```
I       2       5       0.153096
I       5       7       0.459288
I       7       9       0.612384
I       9       11      0.76548
I       11      15      0.918576
I       15      16      1.07167
I       16      17      1.37786
I       17      30      1.68406
```

Epigenomics in a browser

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format

```
I	2	5	0.153096
I	5	7	0.459288
I	7	9	0.612384
I	9	11	0.76548
I	11	15	0.918576
I	15	16	1.07167
I	16	17	1.37786
I	17	30	1.68406
```

```
                                                
```

Epigenomics in a browser

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package

```
> library(rtracklayer)

> import('….bw')

GRanges object with 6243328 ranges and 1 metadata column:
            seqnames    ranges strand |     score
               <Rle> <IRanges>  <Rle> | <numeric>
       [1]        I       3-5      * |  0.153096
       [2]        I       6-7      * |  0.459288
       [3]        I       8-9      * |  0.612384
       [4]        I     10-11      * |  0.765480
       [5]        I     12-15      * |  0.918576
       ...      ...       ...    ... .       ...
  [6243324]     Mito     85775      * |   9.79815
  [6243325]     Mito     85776      * |   8.26719
  [6243326]     Mito     85777      * |   6.43003
  [6243327]     Mito     85778      * |   5.66455
  [6243328]     Mito     85779      * |   3.21502
  -------
  seqinfo: 17 sequences from an unspecified genome
```

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

b b b b k k e e f a a a a a a a a g g g

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

b b b b k k e e f a a a a a a a a g g g

4 b

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

b b b b **k k** e e f a a a a a a a a a g g g

4 b

2 k

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `` `import()` `` from the `` `rtracklayer` `` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
b b b b k k e e f a a a a a a a a g g g
```

```
4 b
2 k
2 e
```

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
b b b b k k e e f a a a a a a a a g g g
```

```
4 b
2 k
2 e
1 f
```

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
b b b b k k e e f a a a a a a a a g g g
```

```
4 b
2 k
2 e
1 f
8 a
```

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
b b b b k k e e f a a a a a a a a g g g
```

```
4 b
2 k
2 e
1 f
8 a
3 g
```

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
b b b b k k e e f a a a a a a a a g g g
```

```
Run-values:  b k e f a g
Run-lengths: 4 2 2 1 8 3
```

Run-length encoding vectors

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
b b b b k k e e f a a a a a a a a g g g
```

```
Run-values:  b k e f a g
Run-lengths: 4 2 2 1 8 3
```

12 alpha-numeric values instead of 20 alphabetic values

Epigenomics in a browser

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
> library(rtracklayer)

> import('….bw')

GRanges object with 6243328 ranges and 1 metadata column:
            seqnames    ranges strand |     score
               <Rle> <IRanges>  <Rle> | <numeric>
       [1]        I       3-5      * |  0.153096
       [2]        I       6-7      * |  0.459288
       [3]        I       8-9      * |  0.612384
       [4]        I     10-11      * |  0.765480
       [5]        I     12-15      * |  0.918576
       ...      ...       ...    ... .       ...
  [6243324]     Mito     85775      * |   9.79815
  [6243325]     Mito     85776      * |   8.26719
  [6243326]     Mito     85777      * |   6.43003
  [6243327]     Mito     85778      * |   5.66455
  [6243328]     Mito     85779      * |   3.21502
  -------
  seqinfo: 17 sequences from an unspecified genome
```

Epigenomics in a browser

- Genomic tracks are generally stored as bigwig files.
- bigwig files store long numerical vectors in a binarized format.
- In R, bigwig files can be imported with `import()` from the `rtracklayer` package.
- bigwig files can be imported as **numerical vectors**, stored as **Run-length encoding vectors**.

```
> import('….bw', as = 'Rle')

RleList of length 17
$I
numeric-Rle of length 230218 with 104639 runs
  Lengths:         2         3         2         2         2 ...         2        30         1      1084
  Values :  0.000000  0.153096  0.459288  0.612384  0.765480 ...  0.612384  0.459288  0.306192  0.000000

$II
numeric-Rle of length 813184 with 424729 runs
  Lengths:         2         1         1         2         1 ...         6         1         5         2
  Values :  0.153096  0.306192  0.612384  0.918576  1.071670 ...  0.459288  0.306192  0.153096  0.000000

...
<15 more elements>
```